

Total Members	1
Members	

Name	Role	Email
Pham Nguyen Khang	Developer	

Overview

Project	Echo Ledger
Track	Track 1 (Agents and strategy) and Track 2 (Dashboards and interfaces)
Problem	Crypto narratives look like consensus when one opinion is reposted many times, and KOL calls are rarely checked against the market. Followers cannot tell independent agreement from an echo, or whether the market backs it.
Solution	An agent that folds reposts into the voice they repeat, scores independent agreement, then checks it against RYO-CHAN's live research (market_overview, sentiment shift, scan_market, deep_analysis, compare_tokens). Deterministic rules decide; every number cites the RYO payload it came from.
Key Features	Evidence ids (E1, E2...) and payload paths on every gate and claim; KOL voices marked corroborated / unconfirmed / contradicted by RYO; vetoes (market risk, late call, weaker than BTC/peer, listed risk); code-computed practice levels (ATR stop, 2R); what-if buttons; replay with the same hash; evidence pack; on-chain card hashes via Chainlink CRE.
Target Users	People who follow crypto KOLs and must decide whether a narrative is worth acting on; researchers and compliance reviewers who need an auditable reason for each decision.
Scope	Public KOL posts (Telegram, X), LLM extraction, echo detection, RYO-CHAN checks, practice decisions only (no real money), dashboard pages /ryo and /judge, Sepolia ledger from the earlier Echo Ledger round.
Limitations	Small sample; practice positions only. Our RYO key was issued with call allowance disabled (allowed: false), so tool calls returned 429 until RYO enables it; the agent records this and decides nothing it cannot back. Telegram terms limit scraping for AI (see README).

Tech Stack

Frontend Stack	React 18, Vite, TypeScript, viem (reads Ethereum Sepolia); static deploy on Vercel.
Backend Stack	Bun + TypeScript agent (ingest, extract, converge, RYO stage); RYO-CHAN REST/MCP client with backoff, throttle and key redaction; Solidity contract (Foundry); Chainlink CRE workflows (record, settle, reputation).
AI Model(s)	OpenAI-compatible LLM (configurable via LLM_MODEL) for post extraction only. Decisions, gates and price levels are deterministic code; no model writes a number.
Other Technologies	RYO-CHAN research layer (6 tools), Chainlink CRE, Ethereum Sepolia, Sourcify, zod.

Repository / Demo

Github Repository	https://github.com/RYO-Digital/ryochan-hackathon_repository-427 (mirror: github.com/nguyenkhangph/echo-ledger-ryo)
Demo Video	Linked in the README (section RYO-CHAN Hackathon 2026 entry) and in the Discord #submit-your-buidl post.
Documentation	README.md (section 'RYO-CHAN Hackathon 2026 entry'); judge page /judge; .env.example lists every variable.

Testing Information

How to run the Project	cp .env.example .env (set RYO_MCP_KEY, LLM_*), then: bun run --cwd agent ingest; extract; converge; bun run ryo:run; bun run ryo:replay; bun run dashboard:dev (open /ryo and /judge).
Prerequisites	Bun 1.3+, Node 20+, a RYO-CHAN MCP key, an OpenAI-compatible LLM key. Optional: Foundry and the Chainlink CRE CLI for the on-chain part.
Installation Steps	git clone <repo>; cd <repo>; bun install
Environment Variables	RYO_MCP_URL, RYO_MCP_KEY, RYO_RATE_PER_MINUTE, LLM_API_KEY, LLM_BASE_URL, LLM_MODEL, CRE_RECORD_URL, VITE_SEPOLIA_RPC_URL, VITE_LEDGER_ADDRESS, VITE_LEDGER_FROM_BLOCK, VITE_SITE_URL (see .env.example)
Build Command	bun run dashboard:build
Run Command	bun run ryo:run (dashboard: bun run dashboard:dev)

Test Command	bun test · bun run ryo:eval · bun run ryo:replay · bun run verify
Test Account(s)	None needed. No login; the dashboard is public. RYO and LLM keys are the tester's own.